Assignment 1 (Focused)

AIL7024
INTRODUCTION TO MACHINE LEARNING
Due: *[08 Sept 2025, 23:59 IST]*

Goal. Build and evaluate, from first principles, **only** the following models: **(i) Linear Regression (MLE)**, **(ii) Logistic Regression**, and **(iii) Decision Trees**.
We provide *function stubs with comments/TODOs*. Implement the methods yourself (no black-box ML).

Rules

- **Allowed:** `numpy`, `pandas`, `matplotlib`, `scikit-learn` *for data management only* (e.g., train/test split, encoders); not models/metrics.
- **Forbidden:** `scikit-learn` models/metrics, `torch`, `tensorflow`, `jax`, `statsmodels`, or any black-box fit/predict utilities.
- Vectorize where feasible; avoid Python loops over samples. Fix and print random seed(s).
- Every figure must have labeled axes and a title. Include units where relevant.

Public datasets (UCI)

Use these three datasets—one per model family. If links change, download manually and place files beside your notebook.

- **Wine Quality (red)** — target: `quality` (separator ";").
- **Banknote Authentication** — target: `class` $\in \{0, 1\}$.
- **Iris** — target: species $\in \{$`setosa`, `versicolor`, `virginica`$\}$.

Required function signatures

Do not change function names or signatures. Replace each `raise NotImplementedError` with your code. Keep docstrings.

Common utilities

```
import numpy as np

def add_intercept(X):
    """
    Return X with a leading column of ones (bias term).
    X: (n_samples, n_features) -> (n_samples, n_features + 1)
    Notes: ensure float dtype; do not modify X in-place.
    """
    raise NotImplementedError

def standardize_fit(X_train, eps=1e-12):
    """
    Standardize TRAIN features to zero mean / unit variance.
```

```
    Return (Xs, mean, std). Use only TRAIN stats.
    """
    raise NotImplementedError

def standardize_apply(X, mean, std, eps=1e-12):
    """
    Apply TRAIN (mean, std) to any new matrix X.
    Do not recompute mean/std here.
    """
    raise NotImplementedError

def train_test_split_idx(n, test_size, seed=0):
    """
    Return (train_idx, test_idx) with fixed RNG.
    You may also use sklearn's splitter to obtain indices only.
    """
    raise NotImplementedError
```

Linear Regression (MLE) — Wine

```
def mse(y, yhat):
    """Mean squared error: average of (y - yhat)^2 (1D arrays)."""
    raise NotImplementedError

def r2_score(y, yhat):
    """1 - SS_res/SS_tot; handle constant-y edge case safely."""
    raise NotImplementedError

def fit_linear_closed_form(X, y, add_bias=True, rcond=None):
    """
    Closed-form least squares (MLE) using pseudoinverse or solve.
    Steps:
      * If add_bias: prepend ones.
      * Prefer pinv for numerical stability when X^T X is ill-conditioned.
    Return w (length p [+1 if bias]).
    """
    raise NotImplementedError

def fit_linear_gd(X, y, lr=0.05, iters=5000, add_bias=True):
    """
    Batch gradient descent for least squares.
    Return (w, losses).
    """
    raise NotImplementedError

def predict_linear(X, w, has_bias=True):
    """Predict with linear model; add bias column if has_bias. Return 1D yhat."""
    raise NotImplementedError
```

Logistic Regression (binary) — Banknote

```
def sigmoid(z):
    """
    Numerically stable logistic sigmoid.
    Tip: clip z to avoid overflow in exp for large |z|.
```

```
    """
    raise NotImplementedError

def logreg_nll(X, y, w, lam=0.0):
    """
    Negative log-likelihood with optional L2 on w[1:].
    NLL = - sum_i [ y_i log p_i + (1-y_i) log (1-p_i) ] + (lam/2)*||w[1:]||^2
    Use epsilon inside logs for stability.
    """
    raise NotImplementedError

def fit_logreg_gd(X, y, lam=0.0, lr=0.1, iters=5000, add_bias=True):
    """
    Gradient descent for logistic regression (L2 on non-intercept).

    Return (w, losses)
    """
    raise NotImplementedError

def fit_logreg_newton(X, y, lam=0.0, iters=100, tol=1e-6, add_bias=True, damping=1e-6):
    """
    Newton's method (IRLS/Fisher scoring).
    Each step solves (H + damping*I) delta = grad; then w <- w - delta.
    Do not regularize intercept. Stop when change < tol.
    """
    raise NotImplementedError

def predict_proba_logreg(X, w, has_bias=True):
    """Return predicted probabilities P(y=1|x)."""
    raise NotImplementedError

def predict_logreg(X, w, threshold=0.5, has_bias=True):
    """Classify by thresholding predicted probabilities."""
    raise NotImplementedError
```

Decision Trees (classification & regression) — Iris & Wine

```
class DTNode:
    """
    Binary decision tree node.
    Recommended attributes:
      feature : int or None      # index of split feature
      threshold : float or None  # split threshold
      left, right : DTNode or None
      value : float/int or None  # leaf prediction (mean or majority)
      is_leaf : bool
    """
    # TODO: implement __init__ and any helpers you need
    raise NotImplementedError

def entropy(y):
    """Shannon entropy (base-2) of labels y."""
    raise NotImplementedError

def gini(y):
    """Gini impurity = 1 - sum_k p_k^2."""
    raise NotImplementedError
```

```python
def mse_impurity(y):
    """Mean squared deviation from mean (for regression leaves)."""
    raise NotImplementedError

def best_split(X, y, criterion="gini", min_samples_leaf=1):
    """
    Return (feat_idx, thr, gain) for the best split.
    Guidance:
      * For each feature f, sort unique values once.
      * Consider midpoints between consecutive distinct values.
      * Skip thresholds that yield < min_samples_leaf in a child.
      * Gain = parent_impurity - weighted_child_impurities.
    """
    raise NotImplementedError

def build_tree_classifier(X, y, max_depth=None, min_samples_split=2,
                          min_samples_leaf=1, criterion="gini", depth=0):
    """
    Greedy top-down induction for classification.
    Stop if: depth hits max_depth, n < min_samples_split, or node is pure.
    Leaf prediction: majority class.
    """
    raise NotImplementedError

def build_tree_regressor(X, y, max_depth=None, min_samples_split=2,
                         min_samples_leaf=1, depth=0):
    """
    Same as classifier but use MSE impurity reduction; leaf predicts mean.
    """
    raise NotImplementedError

def predict_tree(X, tree):
    """Traverse the tree for each sample and return predictions as 1D array."""
    raise NotImplementedError
```

Assignment tasks and deliverables

0) EDA & preprocessing (10 pts)

- Load each dataset. Handle missing values (drop/justify). Print shapes and class/target summaries.
- Compute basic stats (mean/median/std/min/max) for numeric features.
- Train/test split once per dataset (80/20, fixed seeds). Standardize using *train* stats only.
- Produce 2–3 plots per dataset (histograms / pairwise scatter / boxplots) and describe any outliers/skew.

1) Linear Regression via MLE on Wine (25 pts)

- Implement closed-form and GD solvers. Plot GD training loss and discuss convergence behaviour.
- Report MSE and R^2 on train/test; compare coefficient vectors and interpret influential features.

- **Sanity check:** verify GD gradient with finite differences on 3 random coordinates ($\pm 10^{-5}$).

2) Logistic Regression on Banknote (30 pts)

- Train GD and Newton versions with L2 on weights (exclude intercept). Plot NLL vs iterations.
- Report accuracy, precision, recall, F1 on test; include the confusion matrix.
- **Gradient check:** verify analytic gradient on 3 random coordinates via finite differences; report relative error.

3) Decision Trees (35 pts)

- **Classification tree:** Train on *Iris*. Tune `max_depth` and `min_samples_leaf` via 5-fold CV; evaluate test accuracy and the 3×3 confusion matrix. Visualize decision regions in 2D (choose two features; fix others at means).
- **Regression tree:** Train on *Wine* to predict `quality`. Compare test MSE/R^2 vs. linear regression. Discuss when a tree helps/hurts and signs of overfitting.
- **(Optional) Pruning:** Add early-stopping rules; if you implement cost-complexity pruning, show test performance vs. α.

Reproducibility & report (required)

- Centralize seeds; ensure results are reproducible. Include a short (0.5–1 page) discussion comparing models.

Evaluation rubric (100 pts)

Section	Points	What we check
0) EDA & preprocessing	10	Correct splits/standardization, clear plots
1) Linear Regression (MLE)	25	Correctness, convergence plot, gradient check
2) Logistic Regression	30	GD & Newton, metrics, calibration commentary
3) Decision Trees	35	Induction, CV, evaluation, (optional) pruning

Submission

- One notebook (`.ipynb`) and a PDF export with plots.
- A `README.md` with Python/library versions and run instructions.
- Keep function names exactly as specified; we will import for spot tests.

Hints & guardrails

- Use `np.linalg.pinv` or `np.linalg.solve` (watch conditioning). Prefer standardized features.
- For logistic, consider a small damping to Newton's Hessian if ill-conditioned; clip logits for stability.
- For trees, pre-sort per feature or reuse sorted indices to reduce complexity; skip splits with tiny leaves.

- Use consistent random seeds for splits and any sampling; print them in the notebook header.

Academic integrity

Write your own code. Cite any external ideas or references you consult. Submissions may be automatically checked for plagiarism and structural similarity.

End of assignment.